

```

!pip install yfinance
import yfinance as yf
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import random
from collections import deque

```



Collecting yfinance

```

  Downloading yfinance-0.2.57-py2.py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.11/d
Requirement already satisfied: numpy>=1.16.5 in /usr/local/lib/python3.11/d
Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.11/
Collecting multitasking>=0.0.7 (from yfinance)

```

```

  Downloading multitasking-0.0.11-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python
Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.11/di
Collecting frozendict>=2.3.4 (from yfinance)

```

```

  Downloading frozendict-2.4.6-py311-none-any.whl.metadata (23 kB)

```

```

Collecting peewee>=3.16.2 (from yfinance)

```

```

  Downloading peewee-3.18.0.tar.gz (948 kB)

```

949.0/949.0 kB 15.8 MB/s eta 0:

```

Installing build dependencies ... done

```

```

Getting requirements to build wheel ... done

```

```

Preparing metadata (pyproject.toml) ... done

```

```

Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/pyt
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/d
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/p
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/pyt
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/p
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/di
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-p
Downloading yfinance-0.2.57-py2.py3-none-any.whl (113 kB)

```

113.8/113.8 kB 7.7 MB/s eta 0:0

```

Downloading frozendict-2.4.6-py311-none-any.whl (16 kB)

```

```

Downloading multitasking-0.0.11-py3-none-any.whl (8.5 kB)

```

```

Building wheels for collected packages: peewee

```

```

  Building wheel for peewee (pyproject.toml) ... done

```

```

  Created wheel for peewee: filename=peewee-3.18.0-cp311-cp311-linux_x86_64

```

```

  Stored in directory: /root/.cache/pip/wheels/f9/14/97/a6ac3d5b8971bf8ae2b

```

```

Successfully built peewee

```

```

Installing collected packages: peewee, multitasking, frozendict, yfinance

```

```

Successfully installed frozendict-2.4.6 multitasking-0.0.11 peewee-3.18.0 y

```

Double-click (or enter) to edit

```

symbol = "AAPL"
start_date = "2020-01-01"
end_date = "2025-02-14"

```

```
# Download Historical data
data = yf.download(symbol, start=start_date, end=end_date)
display(data.head())
display(data.shape)
display(data.info())
```



YF.download() has changed argument auto_adjust default to True
 [*****100%*****] 1 of 1 completed

Price	Close	High	Low	Open	Volume	
Ticker	AAPL	AAPL	AAPL	AAPL	AAPL	
Date						
2020-01-02	72.716080	72.776606	71.466820	71.721026	135480400	
2020-01-03	72.009132	72.771760	71.783977	71.941343	146322800	
2020-01-06	72.582916	72.621654	70.876083	71.127873	118387200	
2020-01-07	72.241547	72.849224	72.021231	72.592594	108872000	
2020-01-08	73.403633	73.706264	71.943744	71.943744	132079200	

(1287, 5)

<class 'pandas.core.frame.DataFrame'>

DatetimeIndex: 1287 entries, 2020-01-02 to 2025-02-13

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	(Close, AAPL)	1287 non-null	float64
1	(High, AAPL)	1287 non-null	float64
2	(Low, AAPL)	1287 non-null	float64
3	(Open, AAPL)	1287 non-null	float64
4	(Volume, AAPL)	1287 non-null	int64

dtypes: float64(4), int64(1)

memory usage: 60.3 KB

None

```
# Feature Engineering
```

```
data["SMA_5"] = data["Close"].rolling(window=5).mean()
data["SMA_20"] = data["Close"].rolling(window=5).mean()
data["Returns"] = data["Close"].pct_change()
```

```
display(data.head())
display(data.shape)
display(data.info())
```



Price	Close	High	Low	Open	Volume	SMA_5
Ticker	AAPL	AAPL	AAPL	AAPL	AAPL	
Date						
2020-01-02	72.716080	72.776606	71.466820	71.721026	135480400	NaN

```

2020-01-03 72.009132 72.771760 71.783977 71.941343 146322800      NaI
2020-01-06 72.582916 72.621654 70.876083 71.127873 118387200      NaI
2020-01-07 72.241547 72.849224 72.021231 72.592594 108872000      NaI
2020-01-08 73.403633 73.706264 71.943744 71.943744 132079200 72.59066

```

```
(1287, 8)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
DatetimeIndex: 1287 entries, 2020-01-02 to 2025-02-13
```

```
Data columns (total 8 columns):
```

```

#      Column      Non-Null Count  Dtype
---  -
0      (Close, AAPL) 1287 non-null    float64
1      (High, AAPL)  1287 non-null    float64
2      (Low, AAPL)   1287 non-null    float64
3      (Open, AAPL)  1287 non-null    float64
4      (Volume, AAPL) 1287 non-null    int64
5      (SMA_5, )      1283 non-null    float64
6      (SMA_20, )     1283 non-null    float64
7      (Returns, )    1286 non-null    float64

```

```
dtypes: float64(7), int64(1)
```

```
memory usage: 90.5 KB
```

```
None
```

```
# Drop all the null values
```

```
data.dropna(inplace=True)
```

```
data.reset_index(drop=True, inplace=True)
```

```
display(data.head())
```

```
display(data.shape)
```

	Price	Close	High	Low	Open	Volume	SMA_5	SM
	Ticker	AAPL	AAPL	AAPL	AAPL	AAPL		
0		73.403633	73.706264	71.943744	71.943744	132079200	72.590662	72
1		74.962799	75.156480	74.132383	74.384166	170108400	73.040005	73
2		75.132256	75.698786	74.628682	75.197628	140644800	73.664630	73
3		76.737419	76.764054	75.330793	75.449429	121532000	74.495531	74
4		75.701225	76.885120	75.577757	76.674490	161954400	75.187466	75

```
(1283, 8)
```

```
# define action space
```

```
ACTIONS = {0: "Hold", 1: "Buy", 2: "Sell"}
```

```
# extract state from the data
```

```
# get state function
```

```
# def get_state(data, index):
```

```
#     return np.array([
```

```
#         float(data.iloc[index, 'Close']),
```

```
#         float(data.iloc[index, 'SMA_5']),
```

```
#         float(data.iloc[index, 'SMA_20']),
```

```
#         float(data.iloc[index, 'SMA_20']),
#         float(data.iloc[index, 'Returns'])
#     ])

def get_state(data, index):
    return np.array([
        data.iloc[index, data.columns.get_loc('Close']], # Correct positional a
        data.iloc[index, data.columns.get_loc('SMA_5']],
        data.iloc[index, data.columns.get_loc('SMA_20']],
        data.iloc[index, data.columns.get_loc('Returns']]
    ], dtype=np.float32)

# build trading environment for our AI Agent
# define a trading environment to interact with the deep Q-Network(DQN) AI agent

class trading_environment:
    def __init__(self, data):
        self.data = data
        self.initial_balance = 10000
        self.balance = self.initial_balance
        self.holdings = 0
        self.index = 0

    def reset(self):
        self.balance = self.initial_balance
        self.holdings = 0
        self.index = 0
        return get_state(self.data, self.index)

    def step(self, action):
        price = float(self.data.iloc[self.index]['Close'])
        reward = 0

        if action == 1 and self.balance >= price:
            self.holdings = self.balance // price
            self.balance -= self.holdings * price
        elif action == 2 and self.holdings > 0:
            self.balance += self.holdings * price
            self.holdings = 0

        self.index += 1
        done = self.index >= len(self.data)
        if done:
            reward = self.balance - self.initial_balance
            next_state = np.zeros(4) # or handle as appropriate
        else:
            next_state = get_state(self.data, self.index)

        return next_state, reward, done, {}

# deep q network(DQN)
class DQN(nn.Module):
```

```
def __init__(self, state_size, action_size):
    super(DQN, self).__init__()
    self.fc1 = nn.Linear(state_size, 64)
    self.fc2 = nn.Linear(64, 64)
    self.fc3 = nn.Linear(64, action_size)

def forward(self, x):
    # Reshape the input tensor to have the correct dimensions
    x = x.view(x.size(0), -1) # Reshape to (batch_size, state_size)

    x = torch.relu(self.fc1(x))
    x = torch.relu(self.fc2(x))
    return self.fc3(x)

# dqn agent
class DQNAgent:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.memory = deque(maxlen=2000)
        self.gamma = 0.95
        self.epsilon = 1.0
        self.epsilon_decay = 0.995
        self.epsilon_min = 0.01
        self.learning_rate = 0.001
        self.model = DQN(state_size, action_size)
        self.optimizer = optim.Adam(self.model.parameters(), lr=self.learning_rate)
        self.criterion = nn.MSELoss()

    def remember(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))

    def act(self, state):
        if random.uniform(0, 1) <= self.epsilon:
            # return random.choice(list(ACTIONS.keys()))
            return random.randrange(self.action_size)
        state = torch.FloatTensor(state).unsqueeze(0)
        with torch.no_grad():
            q_values = self.model(state)
        return torch.argmax(q_values).item()

    def replay(self, batch_size):
        if len(self.memory) < batch_size:
            return
        minibatch = random.sample(self.memory, batch_size)

        for state, action, reward, next_state, done in minibatch:
            target = reward
            if not done:
                # Reshape next_state to ensure it has the correct dimensions
                # next_state = torch.FloatTensor(next_state).unsqueeze(0)
                next_state = torch.tensor(next_state, dtype=torch.float32).unsqueeze(0)
                target = reward + self.gamma * torch.max(self.model(next_state)).item()
```

```
# state_tensor = torch.FloatTensor(state).unsqueeze(0)
state_tensor = torch.tensor(state, dtype=torch.float32).unsqueeze(0) # Mc
target_tensor = self.model(state_tensor).clone().detach()
target_tensor[0][action] = target

self.optimizer.zero_grad()
output = self.model(state_tensor)
loss = self.criterion(output, target_tensor)
loss.backward()
self.optimizer.step()

if self.epsilon > self.epsilon_min:
    self.epsilon *= self.epsilon_decay

# Train the agent

env = trading_environment(data)
agent = DQNAgent(state_size = 4, action_size = 3)
batch_size = 32
episodes = 100
total_rewards = []

for episode in range(episodes):
    state = env.reset()
    done = False
    total_reward = 0

    while not done:
        action = agent.act(state)
        next_state, reward, done, _ = env.step(action)
        agent.remember(state, action, reward, next_state, done)
        state = next_state
        total_reward += reward

    agent.replay(batch_size)
    total_rewards.append(total_reward)
    print(f"Episode: {episode+1}/{episodes}, Total Reward: {total_reward}")
print("Training Complete")
```

```
<ipython-input-11-7d25906af9ea>:19: FutureWarning: Calling float on a single
price = float(self.data.iloc[self.index]['Close'])
```

```
Episode: 1/100, Total Reward: -9799.905876159668
Episode: 2/100, Total Reward: -9874.92947769165
Episode: 3/100, Total Reward: -9963.888725280762
Episode: 4/100, Total Reward: -9839.548973083496
Episode: 5/100, Total Reward: -9846.879558563232
Episode: 6/100, Total Reward: -9892.983051300049
Episode: 7/100, Total Reward: -9938.36909866333
Episode: 8/100, Total Reward: -9875.380405426025
Episode: 9/100, Total Reward: -9884.914443969727
Episode: 10/100, Total Reward: -9874.928783416748
Episode: 11/100, Total Reward: 3500.899787902832
Episode: 12/100, Total Reward: 6905.287246704102
Episode: 13/100, Total Reward: 9092.47960281372
```

```
Episode: 14/100, Total Reward: -9858.133113861084
Episode: 15/100, Total Reward: -9885.945545196533
Episode: 16/100, Total Reward: -9796.550659179688
Episode: 17/100, Total Reward: -9881.22244644165
Episode: 18/100, Total Reward: -9827.47276687622
Episode: 19/100, Total Reward: -9839.733142852783
Episode: 20/100, Total Reward: -9873.680019378662
Episode: 21/100, Total Reward: -9910.534717559814
Episode: 22/100, Total Reward: -9834.087497711182
Episode: 23/100, Total Reward: -9835.845924377441
Episode: 24/100, Total Reward: -9831.084705352783
Episode: 25/100, Total Reward: -9852.849643707275
Episode: 26/100, Total Reward: -9813.32063293457
Episode: 27/100, Total Reward: -9938.703742980957
Episode: 28/100, Total Reward: -9872.126960754395
Episode: 29/100, Total Reward: -9854.571380615234
Episode: 30/100, Total Reward: -9980.554187774658
Episode: 31/100, Total Reward: -9866.653324127197
Episode: 32/100, Total Reward: -9857.727806091309
Episode: 33/100, Total Reward: -9824.437641143799
Episode: 34/100, Total Reward: -9831.539672851562
Episode: 35/100, Total Reward: -9857.091384887695
Episode: 36/100, Total Reward: -9783.802261352539
Episode: 37/100, Total Reward: -9940.484561920166
Episode: 38/100, Total Reward: -9901.344646453857
Episode: 39/100, Total Reward: -9961.647445678711
Episode: 40/100, Total Reward: -9834.138122558594
Episode: 41/100, Total Reward: -9787.546115875244
Episode: 42/100, Total Reward: 5819.47053527832
Episode: 43/100, Total Reward: -9930.802684783936
Episode: 44/100, Total Reward: -9972.16284942627
Episode: 45/100, Total Reward: -9854.703910827637
Episode: 46/100, Total Reward: -9843.131248474121
Episode: 47/100, Total Reward: 6522.383590698242
Episode: 48/100, Total Reward: -9829.993381500244
Episode: 49/100, Total Reward: -9816.046611785889
Episode: 50/100, Total Reward: -9949.85530090332
Episode: 51/100, Total Reward: 2493.3481407165527
Episode: 52/100, Total Reward: -1840.6021690368652
Episode: 53/100, Total Reward: -9934.049541473389
Episode: 54/100, Total Reward: -9816.831104278564
Episode: 55/100, Total Reward: -9964.843433380127
Episode: 56/100, Total Reward: -9842.345363616943
```

```
# create a fresh environment instance for testing
```

```
test_env = trading_environment(data)
state = test_env.reset()
done = False
```

```
# simulate a trading session using the trained agents
while not done:
```

```
    # always choose the best action (exploitation)
    action = agent.act(state)
    next_state, reward, done, _ = test_env.step(action)
    state = next_state if next_state is not None else state
    print(f"Action: {ACTIONS[action]}, Reward: {reward}")
```

```
final_balance = test_env.balance
profit = final_balance - test_env.initial_balance
print(f"Final Balance after testing: ${final_balance:.2f}")
print(f"Total profit: ${profit:.2f}")
```

```
<ipython-input-11-7d25906af9ea>:19: FutureWarning: Calling float on a single
price = float(self.data.iloc[self.index]['Close'])
```

```
Action: Hold, Reward: 0
Action: Hold, Reward: 0
Action: Hold, Reward: 0
Action: Buy, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Hold, Reward: 0
Action: Buy, Reward: 0
Action: Sell, Reward: 0
Action: Hold, Reward: 0
Action: Hold, Reward: 0
Action: Hold, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Buy, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Hold, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Buy, Reward: 0
Action: Sell, Reward: 0
Action: Buy, Reward: 0
Action: Buy, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Hold, Reward: 0
Action: Hold, Reward: 0
Action: Hold, Reward: 0
Action: Hold, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Buy, Reward: 0
Action: Sell, Reward: 0
Action: Hold, Reward: 0
Action: Buy, Reward: 0
Action: Sell, Reward: 0
Action: Hold, Reward: 0
Action: Hold, Reward: 0
Action: Buy, Reward: 0
Action: Sell, Reward: 0
Action: Sell, Reward: 0
Action: Buy, Reward: 0
Action: Hold, Reward: 0
```



```
Action: Hold, Reward: 0  
Action: Sell, Reward: 0  
Action: Sell, Reward: 0  
Action: Sell, Reward: 0  
Action: Sell, Reward: 0
```

Start coding or [generate](#) with AI.